

Assignment C Report - akali 이연수

Exercise 1. Uranium Finance

사전 지식

Uranium Finance는 Binance Smart Chain에서 운영되던 자동화 마켓메이커(AMM) DEX였다. Uniswap V2를 거의 그대로 포크하면서 수수료 구조만 조금 바꾼 Uniswap 클론 중 하나였다.

Uniswap V2는 K 불변식을 쓴다. 풀 안의 두 토큰 수량을 곱한 값 K가 스왑 후에도 줄어들지 않도록 체크를 한다. 이를 통해 풀에서 돈만 빼가는 공격을 막는다.

원래 Uniswap V2의 swap() 안의 K 체크는

```
require(
    balance0Adjusted.mul(balance1Adjusted) >=
    uint(_reserve0).mul(_reserve1).mul(1000**2),
    'UniswapV2: K'
);
// 여기서 balance0Adjusted = balance0 * 1000 - amount0In * 3
// (0.3% 수수료 계산이 1000 단위로 인코딩됨)
```

이렇게 생겼다. 양변 모두 1000^2 스케일이어서 단위가 딱 맞다.

하지만, Uranium은 수수료를 0.3%에서 0.16%로 낮추려고 리팩토링을 했는데, 좌변은 10000으로 바꿨으면서 우변은 여전히 1000^2 이다.

```
uint balance0Adjusted = balance0.mul(10000).sub(amount0In.mul(16));
uint balance1Adjusted = balance1.mul(10000).sub(amount1In.mul(16));
require(balance0Adjusted.mul(balance1Adjusted) >= uint(_reserve0).mul(_reserve1).mul(1000**2), 'UraniumSwap: K');
```

이는 K 체크가 100배 느슨해져 풀의 99%를 뺏아내도 통과된다.

따라서 풀에 dust input을 미리 보낸 다음 swap을 호출해 양쪽 토큰의 99%를 요청하는 행위를 통해 공격이 가능해졌다.

공격 시도

1. 멘토님 의도 - K 불변식 이용

Uranium Finance 해킹 사건에 쓰인 K 불변식 취약점을 이용하여 공격했다.

사전 지식에서 작성한 바와 같이, 좌변은 10000로 바꿨지만, 우변은 여전히 1000^2 임을 이용해 pair 1개당 두 번의 swap으로 양쪽 reserve의 99%를 뺏아내는 패턴을 이용해 공격을 진행했다.

좌변을 10000으로 바꿨으므로 우변 상수도 10000^2 로 같이 올려야 양변 단위가 맞는다. 이렇게 패치하면 내가 사용했던 공격 방법은 먹히지 않을 것이다.

2. 멘토님의 의도가 아닌 것 - Burger Swap에서 재진입 방어가 없음

Burger Swap이란, 2020년 말에 BSC에 등장한 DEX이다. 내부 엔진이 Demax Platform이라는 컨트랙트다. 에이전트가 Uranium fork를 분석하고 BSC를 탐색하는 과정에서, Uranium Finance 외에도 동일 fork 블록에 Burger Swap이 배포되어 있음을 확인했다.

2021년 5월에 실제로 가짜 토큰을 swap path에 넣어 DEX 내부에서 재진입을 통한 Burger Swap 공격이 있었고, 한화 약 70억원의 피해 규모가 있었다. 원인을 확인해보니, Demax Platform은 멀티 홉 스왑 중 각 토큰의 transferFrom을 호출하는데, 외부 코드를 실행할 수 있다는 점에 대한 재진입 가드가 없었고, 풀 잔액 업데이트가 스왑 완료 시점에 일어나서 콜백 도중에는 이전 값이 그대로 남아있었다.

또한, transferFrom은 토큰 개발자 마음대로 구현이 가능한데, 정상 토큰은 그저 잔액을 옮기지만, 악성 토큰은 그 안에서 무슨 코드든 실행할 수 있도록 한다. 따라서 공격자는 가짜 토큰을 배포해 정상 토큰처럼 path에 들어가게 하고, Demax가 이 가짜 토큰의 transferFrom을 호출하면 가짜토큰의 transferFrom 안에서 공격자가 원하는 코드를 실행할 수 있게 된다.

```
function transferFrom(address from, address to, uint256 amount) external returns (bool) {
    if (from != exploiter) {
        require(allowance[from][msg.sender] >= amount);
        allowance[from][msg.sender] -= amount;
    }
    require(balanceOf[from] >= amount);
    balanceOf[from] -= amount;
    balanceOf[to] += amount;

    if (callbackEnabled) {
        callbackEnabled = false;
        BurgerPoolDrain(payable(exploiter)).onReentrancy();
    }
    return true;
}
```

재진입 코드를 실행함으로써 풀의 상태를 조작할 수 있었다.

이러한 공격을 위해서는 준비 과정이 필요하다. BNB를 WBNB로 래핑하고, 가짜 ERC20 토큰을 발행했다. Demax가 가짜 토큰을 transferFrom할 수 있도록 무한 허용했다. 정상 토큰을 준비해 나중에 반복 회수를 위한 자금을 준비하고, 가짜 토큰과 진짜 토큰으로 pair 2개를 생성했다.

멀티 홉 스왑을 Demax에 요청하면, Demax가 가짜 토큰의 transferFrom을 호출하는 순간 재진입 콜백이 발동된다. 콜백은 앞서 매수해둔 진짜 토큰 전량을 WBNB로 다시 팔아 풀에서 WBNB를 빼왔다. 재진입이 끝나면 바깥 스왑이 그대로 이어져 추가로 WBNB를 받고, 마지막으로 공격 컨트랙트의 WBNB를 unwrap해서 공격자의 EOA로 전송된다.

```
function onReentrancy() external {
    require(msg.sender == address(fakeToken));
    uint256 bal = IERC20(targetToken).balanceOf(address(this));
    if (bal > 0) {
        address[] memory path = new address[](2);
        path[0] = targetToken;
        path[1] = WBNB;
        IDemaxPlatform(PLATFORM).swapExactTokensForTokens(
            bal, 0, path, address(this), block.timestamp + 3600
        );
    }
}
```

Demax는 스왑을 실행할 때 풀 상태를 스왑 시작 시점에 한 번 읽고 출력량을 계산하는데, 콜백 안에서 공격자가 같은 풀을 조작해버리면 Demax가 계산한 출력량과 실제 풀 상태가 어긋나버린다. 이때, Demax는 이 불일치를 적절히 방어하지 못해서 공격자는 같은 유동성을 두 번 뽑아갈 수 있다.

이러한 취약점을 이용한 이유는, 아무리 Uranium Finance의 K 불변식을 이용해서 공격을 해도 1등인 wiker의 점수를 따라잡을 수 없었기 때문이다. 따라서 이러한 공격도 시도해 보았고, 실제로 Uranium Finance K 불변식 취약점을 이용한 공격보다 약 3700 BNB 더 획득할 수 있었다.

이러한 공격을 막으려면 swap 함수에 OpenZeppelin의 nonReentrant modifier를 추가해야한다. 그리고 CEI 패턴을 준수하여 transferFrom 호출 전에 먼저 상태를 업데이트해야 한다.

Exercise 2. Harvest Finance

사전 지식

Harvest Finance는 yield aggregator이다. 사용자가 USDC, USDT와 같은 스테이블코인을 vault에 예치하면 Harvest가 알아서 Curve finance 같은 외부 프로토콜에 재예치해서 수익을 굴려주는 구조였다. 유저는 예치하고 share 토큰을 받는데, 나중에 share 토큰을 돌려주면 원금과 이자를 받는다.

Harvest Vault는 share를 민팅할 때 사용하는 공식에 현재 볼트가 보유한 USDC 총 가치를 계산하는 함수를 포함했다. 이 함수는 Curve Y pool에 재예치된 yUSDC를 얼마로 계산하냐가 문제였다. Harvest는 yUSDC의 가치를 Curve의 calc_withdraw_one_coin로 즉시 조회했는데, Curve pool의 현재 상태가 그대로 Harvest의 share 가격에 반영된다. 외부 오라클을 쓰지 않고 조작 가능한 온체인 가격을 그대로 써버려서, Curve의 순간 가격을 조작하면 Harvest의 fUSDC 민팅량도 조작된다.

실제로 2020년 공격자가 Uniswap V2 Flash Swap으로 수천만 달러를 빌려 Curve Y pool의 비율을 흔들며 Harvest에 예치했고, 풀 복원해서 withdraw로 차익을 뽑는 행위를 반복해 자산을 탈취했다. Harvest 컨트랙트 자체에는 버그가 없었지만, flash loan으로 취약점이 악용됐다.

공격 시도

두 개의 결함이 겹쳐있었다. 하나는 share pricing이 Curve pool의 spot state에 직접 의존해 flash loan 한방에 왜곡된다는 설계 결함이고, 하나는 이를 막기 위한 depositArbCheck 가드가 단일 트랜잭션의 3% 왜곡만 보도록 설계돼 누적 공격을 잡지 못했다. 공격자는 매 반복마다 가드 아래로만 pool을 흔들며 누적치에 대한 체크가 없어 반복 횟수에 제한이 없었다.

Aave V2가 fork 블록에 배포되지 않아 해킹 사건 때와 같은 방법으로 공격할 수는 없었다. 그래서 중첩 플래시 스왑으로 시작해서 큰 청크가 필요할 때는 dYdX Solo Margin을 더해 한 트랜잭션당 14M USDC까지 무수수료로 빌렸다. 그리고 공격을 여러번 나눠 3% 가드 아래로만 Curve pool을 흔들고 반복 횟수를 쌓는 방식을 택했다.

이 취약점은 sharing pricing을 TWAP로 옮겨서 패치하면 좋을 것 같다. Uniswap V3 TWAP oracle이나 Chainlink 가격 피드를 병행해 Curve spot이 외부 기준에서 일정 비율 이상 벗어나면 revert하는 sanity check를 더하면 flash loan 앞에서도 살아남는다. 보조 방어로는 deposit 후 N 블록 이전에는 withdraw를 막는 cooldown 도입이 있다. 같은 트랜잭션에서 갚아야 하는 flash loan 구조와 양립 불가능하므로 공격이 원천 차단된다. 기존 가드를 살리려면 단일 왜곡이 아니라 최근 N 블록의 누적 왜곡을 추적해 revert 하도록 바꾸면 된다.

이 문제는 사실 다른 참가자들이 이득을 지속적으로 많이 봐서 점수가 계속 떨어졌다. 나는 선택과 집중을 해서 Exercise 4번과 5번에서 점수를 많이 먹는 전략을 택했다.

Exercise 3. Fei Rari

사전 지식

Rari Capital은 Compound의 랜딩 프로토콜을 포크해 만든 Fuse라는 서비스를 운영했다. Compound와 다른 점은 누구나 자기 목적에 맞는 랜딩 풀을 permissionless로 만들 수 있다는 점이다. 예금자가 USDC나 DAI 같은 자산을 풀에 예치하면 담보로 잡히면서 f로 시작하는 share 토큰을 받고, 이걸 담보로 다른 자산을 빌릴 수 있다. 2022년 초에 Rari Capital과 Fei Protocol이 합병해서 Fei Rari가 됐다.

이 프로토콜의 취약점은 Compound 원본의 CEther 컨트랙트에 있는 CEI 패턴 위반이었다. 사용자가 ETH를 빌리는 borrow 함수가 doTransferOut을 호출해 공격자에게 ETH를 먼저 보내고 그다음에야 accountBorrows 장부에 빚을 기록한다. ETH를 call.value로 보내면 받는 쪽의 receive 콜백이 모든 가스를 가지고 실행되기 때문에, 장부가 아직 0인 상태에서 공격자는 원하는 컨트랙트 함수를 마음대로 호출할 수 있다.

특히 exitMarket을 호출하면 담보를 잠금 해제할 수 있는데, Fuse의 Comptroller는 이때 accountBorrows를 참조해 빚 유무를 확인했다. 장부가 아직 업데이트되지 않아 빚이 0으로 보이니 exitMarket이 통과되고, 담보는 풀리지만 빌린 ETH는 이미 공격자한테 있는 상태가 된다.

이후 보안 업그레이드에서 주요 함수에 글로벌 reentrancy lock을 붙였다. 하지만 exitMarket에는 lock이 빠져 있었다. 공격자는 같은 트랜잭션 안에서 borrow가 트리거한 receive 콜백 안에서 lock이 걸리지 않은 exitMarket을 호출해서 accountBorrows 업데이트 이전 상태에서 담보 분리에 성공했다.

이후 공격자는 EOA에서 공격 컨트랙트 두 개를 배포한 뒤, 첫 트랜잭션부터 공격을 시작했다. flash loan으로 USDC와 WETH를 빌려 담보로 넣고 fETH에서 ETH를 borrow한 뒤, receive 콜백에서 exitMarket을 호출해 담보를 분리했다. 같은 패턴을 여러 풀에 연달아 적용해 자산을 탈취했다.

공격 시도

두 개의 결함이 있었다. 하나는 Compound 원본에서 물려받은 CEI 위반이다. CEther.borrow가 ETH를 공격자에게 먼저 보낸 뒤에야 accountBorrows를 업데이트하기 때문에 ETH 전송 도중 실행되는 공격자 코드 입장에서는 장부가 아직 0으로 보인다.

하나는 Rari가 보안 업그레이드로 글로벌 reentrancy lock을 붙이면서도 exitMarket에는 lock을 빠뜨린 부분 패치였다. 공격자는 receive 콜백에서 lock이 없는 exitMarket을 호출해 장부가 stale인 동안 담보를 분리했다. 여기에 doTransferOut이 transfer 대신 .call.value를 쓰기 때문에 가스 전체가 콜백으로 넘어가 복잡한 재진입 로직이 실행 가능하다는 전제가 더해진다.

이 취약점에서 기존 사건에서는 한 풀에 집중한 공격이었지만, Fuse는 permissionless라서 살아있는 풀이 많았을거라고 생각했다. 그래서 에이전트를 이용해 전부 스캔해서 각 풀의 CEther 잔액과 borrow 가능 여부를 수집했고, 수집한 리스트를 mask-runner 패턴의 공격 스크립트에 넣어 비트마스크 하나로 이번 트랜잭션에서 털 풀들을 선택할 수 있게 만들었다.

공격 골격은 단순했다. 담보 자산을 ETH로부터 직접 매수하거나 flash loan으로 확보한 뒤 담보 cToken을 mint하고 enterMarkets를 호출한다. 그다음 fETH.borrow를 부르면 doTransferOut이 공격자 컨트랙트의 receive를 트리거하고, receive 안에서 comptroller.exitMarket으로 담보를 잠금 해제한다. 외부 borrow가 리턴될 때쯤엔 담보가 이미 빠진 상태라 cToken을 redeemUnderlying으로 회수하면 빌린 ETH와 담보가 둘 다 손에 남는다. 받은 자산은 Uniswap이나 Balancer를 경유해 ETH로 환전해 다음 풀로 넘어간다.

하지만 과제 실행에서는 풀마다 요구 담보가 달라서 경로를 풀별로 약간씩 바꿨다. 어떤 풀은 DAI 담보만 받아 DAI 한 번 매수로 여러 풀을 순차 드레인하는 재할용이 가능했고, 어떤 풀은 FRAX만 받아 Curve에서 self-fund로 확보해야 했으며, 가장 큰 풀은 stETH와 wstETH를 담보로 받아 Lido와 Balancer를 함께 써야 진입 가능했다. 성공 가능한 풀들을 모두 한 번의 broadcast 번들로 엮어 돌리자 ch3 리더보드에서 잠깐 1등에 올랐다.

다른 경로도 탐색했다. 대부분은 막혔지만 Saddle Finance의 sUSD metapool에서 가격 오차를 이용한 추가 경로 한 가닥을 열어 로컬 잔액을 조금 더 올렸다. 다만 on-chain에 남은 historical max는 CEther 경로의 상한에 가까웠고 최종 점수는 만점 대비 상당한 격차가 남았었다.

하지만 vivald가 의도되지 않은 방식으로 점수를 많이 먹었다고 했다. 이후에 다른 높은 점수를 먹는 참가자들도 나타나 내 점수가 내려갔다. 나도 CEther reentrancy 외의 방법을 탐색했으나 몇몇 큰 풀은 borrow가 원천 차단되거나, fork 블록에 존재했던 다른 프로토콜의 취약점들도 대부분 해당 모듈의 함수 미배포나 refund 경로의 구조적 제약으로 닫혀있어서 실패했다. 따라서 많은 점수를 따지는 못했다.

CEther reentrancy를 막기 위해서는 exitMarket에도 글로벌 reentrancy lock을 붙이는 것이다. 다른 함수에 이미 적용된 lock 체계에 한 줄만 추가하면 콜백 시점의 exitMarket 호출이 차단된다. 그리고 borrow 에서 CEI 순서를 바로 잡아 accountBorrows 업데이트를 ETH 전송 이전으로 옮기면 된다. 이러면 콜백이 실행돼도 장부가 최신이라 exitMarket의 빗검사에 바로 걸린다.

Exercise 4. Superfluid

사전 지식

2022년 공격자가 Superfluid의 host 컨트랙트에 조작된 calldata를 전달해 Super token을 보유한 여러 계정을 스푸핑하는 distribution index를 생성했다. 이 공격으로 많은 자산이 탈취됐고, Qi DAO의 vesting 컨트랙트가 Superfluid 코드를 활용하고 있었기 때문에 가장 큰 피해를 입었다.

Superfluid는 실시간 스트리밍 결제 프로토콜이다. 호스트 컨트랙트는 전체 시스템의 중앙 오케스트라로, 사용자가 callAgreement()로 호출하는 진입점이다. super app이란 여러 Agreement를 조합해서 쓰는 사용자 측 컨트랙트다. Agreement 컨트랙트들은 실제 로직을 가진 모듈로 CFA와 IDA가 대표적이다. CFA는 Contract Flow Agreement로 계속 흐르는 스트림, IDA는 Instant Distribution Agreement로 순간적으로 여러 명에게 배분된다. 이 IDA가 실제 공격에 사용됐다.

한 트랜잭션에서 여러 Agreement가 composable하게 엮일 수 있어야하기 때문에 Superfluid는 ctx라는 직렬화된 상태 객체를 사용한다. ctx에는 agreement 함수가 알아야할 모든 컨텍스트가 담기는데, 특히 초기 호출의 msg.sender가 누구인지에 대한 정보가 포함된다. agreement 컨트랙트 입장에서 호출자가 host 컨트랙트이기 때문에 msg.sender == Host 로 보이기 때문에, ctx를 까봐야 진짜 원래 사용자를 알 수 있다.

원래 정상 흐름은 사용자가 callAgreement를 호출하고, Host가 ctx를 생성해 해시를 _ctxStamp 상태변수에 저장 후 Host가 calldata 안의 placeholder 자리에 ctx를 끼워넣는다. 그리고 Agreement 컨트랙트가 호출돼 ctx를 디코딩해 사용하고, Agreement는 호출자가 인가된 Host인지 확인해야한다.

하지만 이 과정에서 일단 호출한 host가 인증되고 나면, agreement는 전달받은 ctx까지도 신뢰해 이를 메모리 구조체로 디코딩(역직렬화)했다. 사용자는 calldata를 Host에 넘길 때, 나중에 ctx가 채워질

placeholder 자리를 비워둬야하는데, 공격자가 이 placeholder 앞에 자기가 만든 가짜 ctx를 넣어둔 calldata를 보내면 Host는 placeholder 자리에 진짜 ctx를 끼워넣지만 가짜 ctx가 그대로 앞쪽에 남아있게된다. Solidity ABI 디코더는 bytes calldata ctx 파라미터를 읽을 때 가장 먼저 만나는 ctx를 취하고 나머지는 버려서 결국 Agreement는 공격자가 만든 가짜 ctx를 진짜인것 처럼 받아들이게 된다.

이 가짜 ctx 안에 공격자는 msgSender에 피해자 주소를 써서 Agreement는 피해자가 이 작업을 요청했다고 착각하게 한다. 실제 exploit 트랜잭션에서 공격자는 IDA 컨트랙트를 사용해 같은 기법으로 다른 계정들의 자금을 빼냈다.

Superfluid에는 ctx의 유효성을 검증하는 함수가 이미 존재했고, 이 함수는 디코딩된 ctx를 host 컨트랙트에 저장된 해시와 비교해 검증했다. 이 검사는 SuperApp 콜백이 넘겨주는 ctx 데이터에 대해서는 이미 수행되고 있었지만, 신뢰된 Host 컨트랙트가 넘겨주는 데이터에 대해서는 수행되지 않고 있었다. 패치는 단순했는데, Agreement가 ctx를 디코딩한 직후 무조건 해시를 대조하게 했다.

공격 시도

첫 PoC는 피해자 한 명만 지정한 단일 callAgreement 호출로 최소 PoC를 작성해 trailing bytes ctx 위조가 fork 위에서 동일하게 동작하는지 확인했다. msgSender를 피해자 주소로 채운 가짜 ctx를 만들고, IDA claim의 내부 호출을 인코딩하되 마지막 bytes ctx 자리에는 길이 0인 placeholder만 넣은 뒤, 두 조각을 abi.encodePacked로 이어붙여 Host에 넘겼다. Host는 placeholder 길이만 확인하고 그 자리에 진짜 ctx를 덮어써 IDA를 호출했지만, IDA의 ABI 디코더는 먼저 등장하는 가짜 ctx를 읽어 msgSender를 피해자로 오인한 채 settle를 진행했다. 피해자의 분배금이 공격자 주소로 들어오는 걸 확인하면서 primitive가 유효함을 검증했다.

단일 피해자로 원리를 검증한 뒤에는 SuperToken 다섯 종류(USDCx, DAIx, ETHx, MATICx, WBTCx)에서 Transfer 이벤트로 현재 잔고가 양수인 holder를 모아 ctx 위조 체인을 순차 실행했다. 피해자마다 본인 명의의 index를 생성하고, 공격자를 구독자로 등록하고, index 값을 피해자 잔고 전액으로 부풀린 뒤, claim으로 정산했다. MATICx는 downgradeToETH로 바로 native로 풀고, 나머지는 downgrade 후 QuickSwap으로 WMATIC을 거쳐 native로 환산했다. 첫 대규모 drain이었고, 이 시점에 수십만 단위의 MATIC이 확보됐다.

1차 sweep 이후 점수가 정체되는 원인을 파고들어 보니 피해를 만들어내는 방식 자체가 문제였다. 최근 블록의 Transfer 로그에서 후보를 모은 뒤 현재 블록에서 잔고가 양수인 주소만 남기는 방식은 최근 활동이 없는 오래된 holder를 놓친다. 방법을 바꿔 MATICx의 모든 Transfer 이벤트를 최초 블록부터 순회하며 각 주소의 입출금 차액을 직접 누적하는 장부를 재구성했다. 이렇게 복원한 holder 중 잔고가 큰 쪽을 추가로 sweep 대상에 넣자 기존 최고점을 넘었다. 복원된 holder 대부분이 SuperApp이 붙지 않은 순수 EOA라 ctx 위조가 깨끗하게 처리됐고, 이 전수 이력 복원이 이후 모든 피해자 수집의 기본 방식이 됐다.

여기까지는 주요 SuperToken 다섯 종에만 집중하고 있었지만, Superfluid는 누구나 Super Token Factory로 임의의 ERC20을 감싼 커스텀 SuperToken을 만들 수 있는 구조다. factory가 찍은 토큰 목록을 훑어보니 그중 Qix(QI 토큰을 감싼 SuperToken)에 상당량의 QI가 전혀 손대지지 않은 채 쌓여있었다.

2022년 실제 공격에서 Qi DAO가 가장 큰 피해를 본 이유가 QI의 vesting 컨트랙트가 Superfluid IDA 기반이었기 때문인데, 이 포크에도 같은 구조가 그대로 살아있었다. Qix holder를 전수 복원해 일괄 drain한 뒤 뽑아낸 QI를 QuickSwap에서 WMATIC으로 스왑하고 native로 풀었다. 이 한 번의 시도로 총 자산이 두 자릿수 배로 뛰면서 점수를 많이 먹을 수 있었다.

이후에도 점수를 추가로 올리려고 덜 알려진 커스텀 SuperToken(STACKx)과 컨트랙트 주소가 보유한 residual 잔고를 훔쳤다. 마지막 시도에서 STACKx holder 전원과 USDCx/MATICx의 contract residual을 추가로 탈취해 점수를 살짝 더 올렸다. 최종 탈취 자산은 약 360만 MATIC이었다.

실제 패치는 Agreement가 ctx를 디코딩한 직후 해시를 대조하게 만드는 것이었다. 이건 최소 수정이라는 점에서 합리적이지만, 새 Agreement 함수를 추가할 때마다 개발자가 이 검증을 빼먹지 않도록 관리 규율에 의존하는 방식이라 구조적으로는 취약하다. 실제로 이 패치는 v2 문제의 claim 경로에 적용을 누락해서 다시 문제를 일으킨다.

더 근본적인 방안으로는 ctx를 calldata로 주고받는 구조 자체를 없애는 것이라 생각한다. Host가 ctx를 transient storage나 전용 storage slot에 저장하고, Agreement는 필요할 때 Host의 getter로 조회하도록 바꾸면 trailing bytes 공격 자체가 성립하지 않는다. calldata에 계속 실어야 한다면, Agreement의 external 함수마다 require(msg.data.length == expected) 형태로 정확한 calldata 길이를 강제해 trailing bytes를 원천 차단하는 방법도 있다. 또는 ctx 자체에 per-call nonce로 서명을 붙여 Host 바깥에서는 재조립이 불가능하게 만들 수 있다.

Exercise 5. Superfluid_v2

이 문제는 Exercise 4의 trailing bytes ctx 위조를 막기 위해 Agreement 함수 대부분에 authorizeTokenAccess() 호출을 추가했다. 이 함수는 두 가지를 검사한다. 하나는 전달받은 ctx가 실제로 Host가 만든 것인지 해시로 대조하는 것이고, 하나는 호출자가 해당 superToken이 등록된 진짜 Host인지 확인하는 것이다.(token.getHost() == msg.sender) 이 두 검사 덕분에 exercise 4의 trailing bytes 공격은 IDA의 주요 함수에서 전부 invalid ctx 또는 unauthorized host로 막혔다.

이 문제는 IDA.claim()에 authorizeTokenAccess 호출을 빼먹었다. 즉 claim은 caller가 누구인지 검증하지 않는다. 얼핏 보면 ctx 위조가 claim에도 통할 것 같지만, 그것만으로는 자산이 빠지지 않았다. claim의 정산 로직은 ctx의 msgSender나 다른 필드를 읽지 않고 함수 파라미터와 storage 상태만으로 정산 대상과 금액을 결정하기 때문이다. 실제로 ctx 필드를 하나씩 바꿔봐도 정산 결과는 변하지 않았다.

처음에는 exercise4에서 쓰던 trailing bytes ctx 위조 패러다임을 그대로 claim에 가져와 반복했다. ctx 필드를 하나씩 바꿔봤는데 실패했다. ctx가 claim의 정산에 관여하지 않는다는 사실을 뒤늦게 알았다.

SuperApp 콜백을 노리는 경로도 파봤다. Host에 등록된 SuperApp 목록(AppRegistered 이벤트를)을 전수 스캔해 157개를 모았지만, 현재 포크 상태에서 이들이 subscriber로 들어간 IDA index가 단 하나도 없었다.

직접 구독을 심어 콜백을 유도해봐도 real publisher 쪽에서 콜백이 발화되지 않았고, 검증된 publisher 앱의 콜백은 agreementClass가 CFA일 때만 의미 있는 처리를 하고 IDA claim 콜백은 무시하고 바로 정산 로직으로 넘어갔다. 그 외에 CFA 직접 호출, Biconomy trusted forwarder 우회, governance 프록시 탈취 등 여러 개의 공격이 전부 막혔다.

막히던 이유를 다시 정리해보면 caller 검증이 빠진 claim에서 ctx를 위조해도 정산 로직이 ctx를 안 본다는 것이었다. caller 자체를 바꾸면 된다는 것을 깨달았다.

IDA가 claim 실행 중에 호출하는 내부 함수들은 전부 msg.sender를 Host로 간주해 그대로 호출한다. authorizeTokenAccess에서 token.getHost() == msg.sender 체크가 있다면 이 경로를 쓸 수 없지만, claim에는 그 체크가 없다. ISuperfluid 인터페이스만 구현한 공격자 컨트랙트에서 IDA.claim을 직접 호출하면 IDA는 그 컨트랙트를 진짜 Host로 믿고 필요한 콜백을 전부 FakeHost에 넘겨준다.

FakeHost를 배포하고 claim을 직접 호출하는 데까지는 성공했지만, 이것만으로는 토큰이 기존 subscriber에게만 흘러가지 공격자가 가져갈 수 없었다. 여기서 한 단계 더 필요했다.

IDA.claim은 callAppBeforeCallback을 정산 이전에 호출한다. FakeHost가 이 콜백을 제어하므로, 콜백 구현 안에서 다시 claim을 부르면 reentrancy가 성립한다. 이 시점에는 indexValue가 아직 업데이트되지 않은 상태라 같은 pending distribution이 재진입 횟수만큼 반복 정산된다. 3번 재진입하면 4배, 100번 재진입하면 101배가 한 index의 pending 값을 기준으로 정산된다.

그리고 증폭된 금액을 어떻게 공격자 주소로 가져오는지도 문제였다. publisher와 subscriber를 둘 다 공격자가 제어하는 컨트랙트로 두는 self-publish 구조였는데, 공격자 EOA가 publisher로 index를 만들고, 공격자가 배포한 Receiver 컨트랙트를 subscriber로 등록한 뒤 FakeHost로 reentrancy claim을 돌리면, publisher 쪽 잔고는 음수로 빠지지만 subscriber 쪽은 증폭된 양수 잔고를 받는다. 두 주소가 다르고 SuperToken의 정산이 주소별로 독립이기 때문에 subscriber의 양수만 뽑아 downgradeToETH로 native MATIC으로 풀 수 있었다.

첫 성공은 1 MATIC을 넣어 11 MATIC을 뽑은 시도였다. 파라미터를 키워 돌린 본방에서 MATICx 풀 자체의 약 14만 MATIC을 확보했고, 그 뒤로는 다른 SuperToken에도 같은 FakeHost reentrancy를 적용한 뒤 뽑아낸 underlying을 QuickSwap으로 WMATIC 환산했다.

여기서 최적화가 하나 더 있었다. DAIx를 drain해서 나온 DAI를 바로 WMATIC으로 바꾸려 하니 QuickSwap의 DAI/WMATIC 풀 유동성이 9.8만 MATIC 수준밖에 없어 슬리피지 때문에 22만 DAI가 6만

MATIC으로 줄어들었다. 반면 USDC/WMATIC 풀은 800만 MATIC 으로 훨씬 높았다. 그래서 경로를 DAI를 먼저 USDC로 바꾸고 그다음 WMATIC으로 가는 멀티 hop 방식으로 바꿨더니 같은 잔고에서 13.5만 MATIC이 나와 2.25배 가까이 회수율이 올랐다.

이후 QIX와 SUSHIX에도 같은 FakeHost reentrancy를 적용해 추가로 탈취했다. QIX에서는 약 10만 MATIC이 더 나왔다. RICX는 `getUnderlyingToken()`이 0 주소를 반환해 native 환산이 불가능해 제외했다. 최종 잔고는 약 100만 MATIC였다.

만점에서 약 760점이 모자란 이유는 두 가지다. 첫째, DAIx 잔여 컨트랙트 일부는 claim 경로 자체는 열려 있어도 subscriber가 컨트랙트라 downgrade 단계에서 거부됐다. 둘째, 일부 커스텀 SuperToken은 underlying의 QuickSwap 유동성이 너무 얇아 super-token 단위로는 많이 뽑아도 native 환산 시 거의 증발했다.

IDA.claim()의 진입부에 `authorizeTokenAccess()` 호출을 추가하면 `token.getHost() == msg.sender` 체크가 걸려 FakeHost가 차단된다. 이렇게 패치하면 될 것 같다.

하지만 진짜 문제는 claim의 caller 검증 누락 하나가 아닌, 정산 이전에 외부로 빠져나가는 콜백이 존재한다는 문제가 존재한다. 이를 막으려면 두 가지 방어를 동시에 해야한다. 하나는 IDA의 모든 external 함수에 `nonReentrant` modifier를 걸어 `authorizeTokenAccess`가 빠진 경우에도 reentrancy 자체가 성립하지 않게 하는 것이고, 하나는 claim 내부 로직을 CEI 순서로 재배치해 정산이 외부 콜백보다 먼저 끝나게 하는 것이다. 이 두 가지가 같이 적용되면 비슷한 실수가 다른 함수에서 재발해도 자산 탈취까지 이어지지 않는다.

가장 강력한 방어는, Host 검증을 함수별 수동 체크에 의존하지 말고 Agreement 컨트랙트의 공통 modifier나 proxy dispatcher 수준에서 방어해야한다.

AI 활용

과제 B는 하네스 엔지니어링을 수행했지만, hook을 강력하게 구축하지 않았고, claude code cli 하나만을 활용해서 모든 작업을 병렬로 진행하게 했다. 그래서 성능을 극대화하지 못하고, 결국 속도도 성능도 좋은 상태를 유지하지 못했었다.

이번 과제 C에서는 agent 하네스 엔지니어링 공부를 조금 더 해봤다. 완전 자동화와 멀티 에이전트를 사용해 역할 분담을 했다. claude code cli는 판단, 추론 등 두뇌가 필요한 일을 진행시켰고, codex를 exploit, 분석, 코딩 등 비교적 생각을 덜 해도되거나 추론이 많이 필요가 없는 잡일들을 맡겼다. codex가 분석해오면 claude가 그 자료를 바탕으로 추론 및 판단을 해주고, codex는 claude의 자료를 바탕으로 exploit을 진행하도록 했다. 이러면 조금 더 빠르게 소모되는 claude의 토큰을 아낄 수 있었다. (만약

claude와 codex 둘 다 없었다면 opus 모델과 sonnet 모델로 멀티에이전트를 구축하거나, 더 저렴하고 빠른 모델을 사용해 잡일을 시켰을 것이다.)

이렇게 멀티 에이전트를 구축하니까 역할 분담을 통해 claude의 추론 능력이 저번 과제 때보다 더 향상된 것 같았다. 그래서 exercise 5 점수를 관찮게 먹을 수 있었던 것 같다.

또한, review 에이전트를 구축하여 에이전트들의 성능을 극대화시켰다. llm은 자기객관화를 못하고 칭찬하는 경향이 있다. 따라서, review 에이전트를 통해 제 3자가 해주는 평가를 바탕으로 두뇌 에이전트가 객관화를 잘 할 수 있도록 했다.

knowledge를 미리 작성해두고 에이전트들이 조금 더 기본적인 지식들을 더 많이 키울 수 있게 했다. knowledge에는 수업시간에 했던 realworld 사례들과 과제 C에 대한 규칙들, 그리고 사건들마다 과거에 어떻게 공격당했고 어떻게 패치했는지에 대한 정보들을 담았다.

저번 과제 B보다 이번 과제 C에서 작성한 skill들은 조금 더 자세하지만 짧게 작성했다. 너무 길면 말을 잘 듣지 않는 것 같아서 짧게 작성했다.

그리고 exercise 1,2,3,4와 exercise 5 문제의 난이도가 차이가 났기 때문에, 터미널 두 개를 열어 세션을 나눠서 에이전트 두 개로 돌렸다. 세션은 여러 개를 돌려도 분리되어 있어 충돌이 없다는 것을 최근에 알게 됐고, 이 때문에 성능이 낮아질 일이 없다는 것을 알게 됐다. 아마 성능이 낮아진다면, CPU를 너무 많이 먹어서 속도가 느려지는 이유 밖에 없다. 따라서 이 또한 추론 능력을 높일 수 있었다고 본다.

느낀점

exercise 1번에서 멘토님의 의도가 아닌 취약점을 이용해 점수를 획득했는데, 이 또한 학습할 수 있는 기회라 생각해서 감사히 공부했다. exercise 5번 또한 멘토님의 의도가 아니었던 취약점을 이용해서 점수를 획득했었는데, 아무리 분석해도 이거 말고 더 어떤 요소로 점수를 얻을 수 있는지 모르겠다. 이 부분에 대해서는 다른 참가자들의 발표를 들어보고 싶다고 생각했다. 그리고 이번에는 저번보다 하네스 엔지니어링을 좀 더 제대로 했던 것 같아서 더 배우고 에이전트 장인이 되고싶다고 느꼈다.

또한, 에이전트가 편한 이유 중 하나는 많고 긴 코드에서 취약해보이는 코드가 어디있는지를 금방 찾아줄 수 있어서 빠르게 가서 확인해볼 수 있다는 점이다. 이 덕분에 잘 수행하고 있는지를 알 수 있었다. 잘 못 수행하고 있다면 claude가 제대로 판단하지 않았거나 hook을 제대로 걸지 않아 claude가 판단을 제대로 안했기 때문에 에이전트의 성능을 조금씩 늘릴 수 있는 연습을 할 수 있었다.